Моки, стабы и изоляция зависимостей

1. Внешние зависимости

Корректный unit-тест обязан быть **детерминированным**: при одинаковых входных данных он должен давать одинаковый результат вне зависимости от машины, окружения и момента запуска. Но внешние зависимости нарушают это требование.

Зависимость — это любой объект или ресурс, который влияет на выполнение кода, но не является частью проверяемой логики. Такими зависимостями могут выступать: файл, в который сохраняются данные; системные часы, под временем которых создаётся запись; имя пользователя, полученное из переменных окружения.

Пока зависимость жёстко встроена внутрь метода, тест не может её контролировать — а значит, не может гарантировать предсказуемость результата. Рассмотрим далее некоторые примеры внешних зависимостей и к каким проблемам они могут привести.

Файловая система

Файл, оставшийся от предыдущего запуска, способен повлиять на результат следующего теста. Два теста, запущенных параллельно, могут обращаться к одному и тому же файлу и нарушать работу друг друга. Может отсутствовать необходимая директория или не быть прав на запись. Поведение файловых путей на Windows и Linux различается, что порождает дополнительную непредсказуемость.

Рассмотрим простой сервис сохранения задач:

```
public void SaveTasks(List<TaskItem> tasks)
{
    File.WriteAllLines("tasks.txt", tasks.Select(t => t.ToString()));
}
```

Тест для подобного метода неизбежно зависит от состояния диска. Это означает, что он проверяет не логику метода, а поведение операционной системы.

Текущее время

Время является одним из источников нестабильности и проблема, как правило, проявляется не сразу. Рассмотрим пример:

```
public bool IsTaskOverdue(TaskItem task)
{
    return task.Deadline < DateTime.Now;
}
```

Написать предсказуемый тест для такого метода невозможно. Значение `DateTime.Now` меняется при каждом запуске. Логика метода при этом может быть абсолютно верной. Проблема состоит в том, что `DateTime.Now` представляет собой скрытую зависимость от системных часов, которую тест не контролирует.

Сетевые вызовы

HTTP-запрос в unit-тесте означает зависимость от сторонней системы, поведение которой тест не контролирует никак.

```
public async Task<string> GetUserNameAsync(int userId)
{
    var response = await
_httpClient.GetAsync($"https://api.example.com/users/{userId}");
    var json = await response.Content.ReadAsStringAsync();
    return JsonSerializer.Deserialize<User>(json).Name;
}
```

Внешний сервис может быть недоступен, отвечать с задержкой, вернуть изменённый формат данных после обновления API или требовать токен аутентификации, который присутствует в локальной среде, но отсутствует на сервере, на котором проводится тестирование. Тест, зависящий от сети, фактически тестирует чужой код. Unit-тест должен проверять исключительно ту логику, которая реализована в рамках проекта.

2. Тестовые двойники: stub, fake, mock

Для того, чтобы можно было тестировать код с внешними зависимостями, используются объекты, замещающие реальные зависимости в тестах, они называются **тестовыми двойниками**. Существует несколько их разновидностей: **stub, fake и mock** каждый вид решает собственную задачу.

Механизм подмены зависимости

Прежде чем рассматривать виды двойников, необходимо разобраться с тем, как подмена становится возможной. Пока зависимость создаётся непосредственно внутри метода, заменить её не представляется возможным:

```
public class TaskService
{
    public void SaveTasks(List<TaskItem> tasks)
    {
        File.WriteAllLines("tasks.txt", tasks.Select(t => t.ToString()));
    }
}
```

Здесь `File` — статический класс, жёстко вписанный в логику метода. Тест не имеет возможности указать сервису использовать иной объект вместо него.

Чтобы подмена стала возможной, зависимость необходимо вынести за пределы метода и передавать извне. Стандартный способ — внедрение через конструктор:

```
public interface ITaskStorage
{
    void Save(IEnumerable<TaskItem> tasks);
}

public class TaskService
{
    private readonly ITaskStorage _storage;

    public TaskService(ITaskStorage storage)
    {
        _storage = storage;
    }

    public void SaveTasks(IEnumerable<TaskItem> tasks)
    {
        _storage.Save(tasks);
    }
}
```

Теперь сервис не осведомлён о конкретном способе сохранения данных — он взаимодействует с интерфейсом, а не с реализацией. В тесте можно передать любой объект, реализующий `ITaskStorage`, что и открывает возможность для применения тестовых двойников.

Stub

Stub — простейшая разновидность тестового двойника. Он применяется в тех случаях, когда тестируемому коду требуется некоторый результат от зависимости, однако способ получения этого результата не является предметом проверки.

Stub возвращает заранее заданные данные и не выполняет никаких дополнительных действий. Он не содержит логики и не фиксирует факты вызовов.

```
public interface IUserContext
{
    string CurrentUser { get; }
}

public class StubUserContext : IUserContext
{
    public string CurrentUser => "test_user";
}
```

Применение в тесте:

```
[Fact]
public void AddTask_UserExists_TaskIsAddedForUser()
{
    var userContext = new StubUserContext();
    var service = new TaskService(userContext);

    service.AddTask("Task");

    Assert.Equal("test_user", service.Tasks.First().Owner);
}
```

Stub создаёт необходимое состояние для теста. Количество обращений к свойству `CurrentUser` не представляет интереса — важно лишь то, что сервис получил корректное значение и правильно с ним поработал.

Stub является правильным выбором в тех случаях, когда тест проверяет состояние, а не взаимодействие.

Fake

Fake — тестовый двойник с реальной рабочей логикой, упрощённой по сравнению с настоящей реализацией. Наиболее классический пример — хранилище данных в оперативной памяти, замещающее файловую систему или базу данных.

```
public class InMemoryTaskStorage : ITaskStorage
{
    public List<TaskItem> SavedTasks { get; } = new();

    public void Save(IEnumerable<TaskItem> tasks)
    {
        SavedTasks.AddRange(tasks);
    }
}
```

Применение в тесте:

```
[Fact]
public void SaveTasks_Called_TasksAreStored()
{
    var storage = new InMemoryTaskStorage();
    var service = new TaskService(storage);

    service.SaveTasks(new[] { new TaskItem("Task") });

    Assert.Single(storage.SavedTasks);
}
```

В отличие от stub, fake действительно выполняет работу: объекты реально добавляются в список. При этом не задействуются ни файлы, ни база данных, ни какая-либо инфраструктура.

Fake уместен в тех случаях, когда зависимость достаточно сложна, чтобы обладать собственным поведением, и когда необходимо проверить несколько последовательных взаимодействий.

Mock

Mock — тестовый двойник, проверяющий не то, что вернула зависимость, а то, как именно с ней взаимодействовали: был ли вызван метод, сколько раз и с какими аргументами.

Mock необходим в тех случаях, когда сам факт взаимодействия является частью проверяемого поведения.

```
public class MockTaskStorage : ITaskStorage
{
    public int SaveCallCount { get; private set; }

    public void Save(IEnumerable<TaskItem> tasks)
    {
        SaveCallCount++;
    }
}
```

```
[Fact]
public void SaveTasks_Called_StorageSaveInvokedOnce()
{
    var mockStorage = new MockTaskStorage();
    var service = new TaskService(mockStorage);

    service.SaveTasks(new List<TaskItem>());

    Assert.Equal(1, mockStorage.SaveCallCount);
}
```

В данном тесте результат выполнения метода не важен. Предметом проверки является соблюдение контракта взаимодействия: был ли вызов совершён ровно один раз.

На практике моки редко реализуются вручную — для этого используются специализированные фреймворки. Однако понимание лежащей в основе идеи необходимо прежде, чем переходить к инструментарию.

Критерии выбора тестового двойника

Выбор между разновидностями двойников определяется ответом на один вопрос: что именно проверяет данный тест?

Если тест проверяет **состояние** — что получилось в результате работы метода — как правило, достаточно stub или fake. Если тест проверяет **взаимодействие** — был ли совершён вызов и с какими параметрами — необходим mock.

Задача	Двойник
Вернуть данные, собственная логика отсутствует	Stub
Рабочая логика без инфраструктуры	Fake
Проверить факт и параметры вызова	Mock

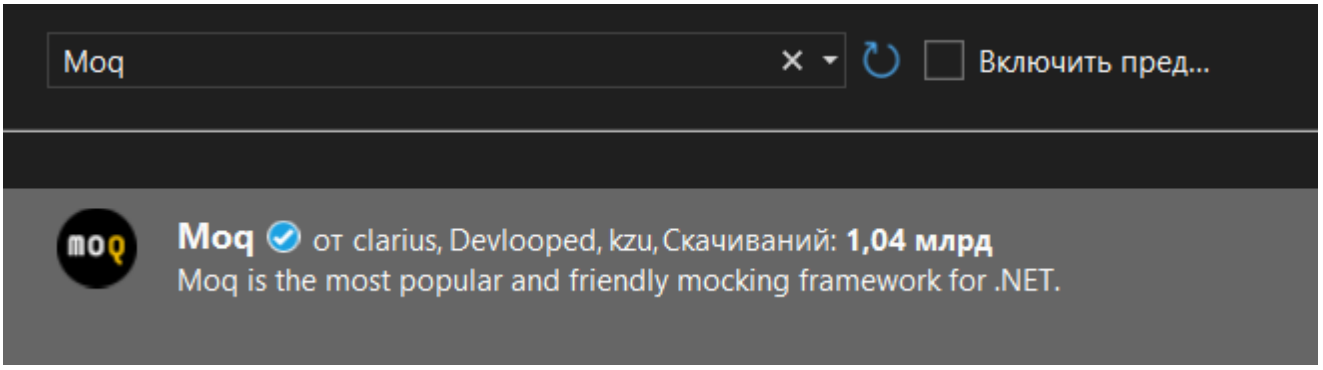
3. Moq

В предыдущем разделе тестовые двойники реализовывались вручную: для каждой зависимости создавался отдельный класс, реализовывался интерфейс, добавлялись счётчики вызовов. Такой подход работоспособен, однако быстро становится трудоёмким. При значительном количестве зависимостей объём вспомогательного кода начинает превышать объём самих тестов.

Moq — это библиотека, позволяющая создавать моки и стабы непосредственно в теле теста, без объявления отдельных классов. Всё, что ранее реализовывалось вручную, выражается в нескольких строках декларативного кода.

Установка

Moq подключается через NuGet в тестовый проект:



либо через командную строку:

```
dotnet add package Moq
```

После установки достаточно добавить соответствующую директиву в файл с тестами:

```
using Moq;
```

Создание мока

Мок создаётся через обобщённый класс `Mock<T>`, где `T` — интерфейс, подлежащий подмене:

```
var storageMock = new Mock<ITaskStorage>();
```

Объект-двойник, передаваемый в тестируемый класс, извлекается через свойство `.Object`:

```
var service = new TaskService(storageMock.Object);
```

По умолчанию все методы мока не выполняют никаких действий и возвращают значения по умолчанию: `null` для ссылочных типов, `0` для числовых, `false` для булевых. В этом состоянии мок ведёт себя как stub — реальный код зависимости не вызывается.

Настройка возвращаемого значения: Setup и Returns

Если тестируемому коду требуется конкретный результат от зависимости, используется метод `Setup` в сочетании с `Returns`:

```
var repositoryMock = new Mock<ITaskRepository>();

repositoryMock
    .Setup(r => r.GetAll())
    .Returns(new List<TaskItem> { new TaskItem("Task") });
```

После такой настройки при каждом вызове `GetAll()` мок будет возвращать заданный список. Тестируемый код получает предсказуемые данные, не обращаясь ни к файлу, ни к базе данных.

Полный тест:

```
[Fact]
public void GetTasks_ReturnsTasksFromRepository()
{
    // Arrange
    var repositoryMock = new Mock<ITaskRepository>();
    repositoryMock
        .Setup(r => r.GetAll())
        .Returns(new List<TaskItem> { new TaskItem("Task") });

    var service = new TaskService(repositoryMock.Object);

    // Act
    var result = service.GetTasks();

    // Assert
    Assert.Single(result);
}
```

Сопоставление аргументов: It

Если нужно проверить, что метод был вызван с аргументом подходящего типа — используется вспомогательный класс `It`:

- `It.IsAny<T>()` — принимает любое значение указанного типа;
- `It.Is<T>(predicate)` — позволяет задать произвольное условие через лямбда-выражение.

```
// Любой список задач
storageMock.Setup(s => s.Save(It.IsAny<IEnumerable<TaskItem>>()));

// Только непустые строки
repositoryMock.Setup(r => r.Find(It.Is<string>(s =>
!string.IsNullOrEmpty(s))));
```

Проверка вызовов: Verify

Если предметом проверки является не возвращаемое значение, а сам факт взаимодействия с зависимостью, используется метод `Verify`:

```
storageMock.Verify(s => s.Save(It.IsAny<IEnumerable<TaskItem>>()),
Times.Once);
```

Данная строка утверждает, что метод `Save` был вызван ровно один раз. Если условие не выполнено — тест завершается с ошибкой.

Класс `Times` предоставляет набор предикатов для точного описания ожидаемого количества вызовов:

Предикат	Значение
`Times.Once`	Ровно один раз
`Times.Never`	Ни разу
`Times.Exactly(n)`	Ровно n раз
`Times.AtLeastOnce`	Хотя бы один раз
`Times.AtMost(n)`	Не более n раз

`Times.Never` полезен для проверки того, что при определённых условиях вызов вообще не должен происходить:

```
[Fact]
public void SaveTasks_EmptyList_StorageIsNotCalled()
{
    // Arrange
    var storageMock = new Mock<ITaskStorage>();
    var service = new TaskService(storageMock.Object);

    // Act
    service.SaveTasks(new List<TaskItem>());

    // Assert
    storageMock.Verify(s => s.Save(It.IsAny<IEnumerable<TaskItem>>()),
Times.Never);
}
```

Проверка параметров вызова

`Verify` допускает проверку не только факта вызова, но и конкретных аргументов, с которыми он был совершён:

```
storageMock.Verify(
    s => s.Save(It.Is<IEnumerable<TaskItem>>(
        tasks => tasks.Any(t => t.Text == "New Task"))),
    Times.Once
);
```

Такой тест утверждает, что метод `Save` был вызван один раз, и переданная коллекция содержала элемент с заданным текстом.

Генерация исключений: Throws

Moq позволяет настроить мок таким образом, чтобы при вызове метода выбрасывалось исключение. Это необходимо для проверки того, как тестируемый код обрабатывает сбои зависимостей:

```
storageMock
    .Setup(s => s.Save(It.IsAny<IEnumerable<TaskItem>>()))
    .Throws(new IOException("Disk is full"));
```

Тест на обработку такого сбоя:

```
[Fact]
public void SaveTasks_StorageFails_ThrowsException()
{
    // Arrange
    var storageMock = new Mock<ITaskStorage>();
    storageMock
        .Setup(s => s.Save(It.IsAny<IEnumerable<TaskItem>>()))
        .Throws(new IOException("Disk is full"));

    var service = new TaskService(storageMock.Object);

    // Act & Assert
    Assert.Throws<IOException>(() => service.SaveTasks(new List<TaskItem>
()));
}
```

4. Мокирование времени и асинхронных методов

Часть 1. Мокирование времени

Проблема DateTime.Now

Системное время представляет собой один из наиболее распространённых источников нестабильности в unit-тестах. Метод, напрямую обращающийся к `DateTime.Now`, содержит скрытую зависимость, которую тест не может контролировать:

```
public void AddTask(string text)
{
    var task = new TaskItem(text)
    {
        CreatedAt = DateTime.Now
    };
    _storage.Save(task);
}
```

Написать детерминированный тест для такого метода невозможно: значение `DateTime.Now` меняется при каждом запуске, и зафиксировать его средствами теста нельзя.

Интерфейс IClock

Решение состоит в том, чтобы вынести зависимость от времени за пределы метода и передавать её через интерфейс — по тому же принципу, что и любую другую зависимость:

```
public interface IClock
{
    DateTime Now { get; }
}
```

Рабочая реализация интерфейса делегирует вызов системным часам:

```
public class SystemClock : IClock
{
    public DateTime Now => DateTime.Now;
}
```

Тестируемый класс принимает `IClock` через конструктор и использует его вместо прямого обращения к `DateTime.Now`:

```
public class TaskService
{
    private readonly ITaskStorage _storage;
    private readonly IClock _clock;

    public TaskService(ITaskStorage storage, IClock clock)
    {
        _storage = storage;
        _clock = clock;
    }

    public void AddTask(string text)
    {
        var task = new TaskItem(text)
        {
            CreatedAt = _clock.Now
        };
        _storage.Save(task);
    }
}
```

Теперь зависимость от времени явная, передаётся извне и может быть подменена в тесте.

Мокирование IClock через Moq

В тесте мок `IClock` настраивается на возврат фиксированного значения:

```
[Fact]
public void AddTask_Called_TaskCreatedAtFixedTime()
{
    // Arrange
    var fixedTime = new DateTime(2025, 1, 1, 12, 0, 0);

    var clockMock = new Mock<IClock>();
    clockMock.Setup(c => c.Now).Returns(fixedTime);

    var storageMock = new Mock<ITaskStorage>();
    var service = new TaskService(storageMock.Object, clockMock.Object);

    // Act
    service.AddTask("Task");

    // Assert
    storageMock.Verify(s => s.Save(
        It.Is<TaskItem>(t => t.CreatedAt == fixedTime)),
        Times.Once); }
```

Тест полностью детерминирован: время зафиксировано, результат предсказуем, поведение теста не зависит от момента запуска и среды выполнения.

Мокирование проверки актуальности задачи

Рассмотрим ещё один случай — проверку того, является ли задача просроченной:

```
public bool IsTaskOverdue(TaskItem task)
{
    return task.Deadline < _clock.Now;
}
```

После внедрения `IClock` тест для данного метода становится тривиальным:

```
[Fact]
public void IsTaskOverdue_DeadlineInPast_ReturnsTrue()
{
    // Arrange
    var fixedTime = new DateTime(2025, 6, 1);

    var clockMock = new Mock<IClock>();
    clockMock.Setup(c => c.Now).Returns(fixedTime);
    var storageMock = new Mock<ITaskStorage>();

    var service = new TaskService(storageMock.Object, clockMock.Object);
    var task = new TaskItem("Task") { Deadline = new DateTime(2025, 5, 1) };

    // Act
    var result = service.IsTaskOverdue(task);

    // Assert
    Assert.True(result);
}
```

Дедлайн задачи установлен в мае, зафиксированное «текущее время» — июнь. Результат будет неизменно равен `true` вне зависимости от того, когда запускается тест.

Часть 2. Асинхронные методы

Асинхронность в интерфейсах

Современные приложения широко используют `async/await`. Методы репозиториев, сервисов хранения и других зависимостей нередко возвращают `Task` или `Task<T>`. Принципы тестирования при этом остаются прежними, однако синтаксис работы с Moq имеет ряд особенностей.

Предположим, интерфейс хранилища задач стал асинхронным:

```
public interface ITaskRepository
{
    Task<IEnumerable<TaskItem>> GetAllAsync();
    Task SaveAsync(IEnumerable<TaskItem> tasks);
}
```

ReturnsAsync и Returns(Task.CompletedTask)

Для асинхронных методов, возвращающих `Task<T>`, вместо `Returns` используется `ReturnsAsync`:

```
var repositoryMock = new Mock<ITaskRepository>();

repositoryMock
    .Setup(r => r.GetAllAsync())
    .ReturnsAsync(new List<TaskItem> { new TaskItem("Task") });
```

Для методов, возвращающих `Task` без результата, применяется `Returns(Task.CompletedTask)`:

```
repositoryMock
    .Setup(r => r.SaveAsync(It.IsAny<IEnumerable<TaskItem>>()))
    .Returns(Task.CompletedTask);
```

Тест с async/await

Тест для асинхронного метода сервиса должен быть объявлен как асинхронный: возвращать `Task` и использовать `await` при вызове тестируемого кода:

```
[Fact]
public async Task GetTasks_ReturnsTasksFromRepository()
{
    // Arrange
    var repositoryMock = new Mock<ITaskRepository>();
    repositoryMock
        .Setup(r => r.GetAllAsync())
        .ReturnsAsync(new List<TaskItem> { new TaskItem("Task") });

    var service = new TaskService(repositoryMock.Object);

    // Act
    var result = await service.GetTasksAsync();

    // Assert
    Assert.Single(result);
}
```

Проверка вызова асинхронного метода

`Verify` применяется для асинхронных методов так же, как и для синхронных:

```
[Fact]
public async Task SaveTasks_Called_RepositorySaveInvokedOnce()
{
    // Arrange
    var repositoryMock = new Mock<ITaskRepository>();
    repositoryMock
        .Setup(r => r.SaveAsync(It.IsAny<IEnumerable<TaskItem>>()))
        .Returns(Task.CompletedTask);

    var service = new TaskService(repositoryMock.Object);

    // Act
    await service.SaveTasksAsync(new List<TaskItem>());

    // Assert
    repositoryMock.Verify(
        r => r.SaveAsync(It.IsAny<IEnumerable<TaskItem>>()),
        Times.Once
    );
}
```

Создание исключений в асинхронном коде

Для проверки обработки сбоев асинхронной зависимости используется `ThrowsAsync`:

```
repositoryMock
    .Setup(r => r.GetAllAsync())
    .ThrowsAsync(new InvalidOperationException("Storage unavailable"));
```

Тест на корректную обработку такого сбоя:

```
[Fact]
public async Task GetTasks_RepositoryFails_ThrowsException()
{
    // Arrange
    var repositoryMock = new Mock<ITaskRepository>();
    repositoryMock
        .Setup(r => r.GetAllAsync())
        .ThrowsAsync(new InvalidOperationException("Storage unavailable"));

    var service = new TaskService(repositoryMock.Object);

    // Act & Assert
    await Assert.ThrowsAsync<InvalidOperationException>(
        () => service.GetTasksAsync()
    );
}
```

Практическое задание

Что нужно сделать:

1. Подготовка
   Установите в тестовом проекте `TodoList.Tests` пакет Moq.
2. Создайте интерфейс `IClock`
   В основном проекте создайте файл `IClock.cs`:

   ```
   public interface IClock
   {
       DateTime Now { get; }
   }
   ```

3. Создайте реализацию `SystemClock`
   В основном проекте создайте файл `SystemClock.cs`, который реализует `IClock` и возвращает реальное системное время.
4. Измените класс `TodoItem`

   Класс `TodoItem` использует `DateTime.Now` напрямую — это скрытая зависимость, которую тест не может контролировать. Ваша задача:
   1. Добавьте `IClock` как параметр конструктора.
   2. Установите `SystemClock`, как значение по умолчанию в конструкторе, чтобы не переписывать код в других местах.
   3. Замените все обращения к `DateTime.Now` на вызов `_clock.Now`.
   4. Убедитесь, что существующее поведение не изменилось.
5. Напишите unit-тесты
   В `TodoItemTests` добавьте тесты, которые проверяют правильность изменения параметра `LastUpdate` при создании и изменении задачи. Время в тестах фиксируйте через мок `IClock`. Не используйте `DateTime.Now` в тестах. Используйте паттерн AAA для написания тестов. Именуйте тесты по схеме `MethodName_Scenario_ExpectedResult`.

В качестве дополнительного задания можете реализовать тестирование для класса `FileManager` с использование библиотеки Moq. За это будут выставлены дополнительные баллы.